PYTHON FOR ACTUARIES · NOTEBOOK 08 · SOLUTION

GLMs for Actuaries

A 90-Minute Practical Workshop

> **Disclaimer:** This notebook estimates the **average claim amount** (claim severity), not the premium. A premium also needs the **claim frequency**: how many claims a contract causes per year. That requires all contracts, including those without a claim, and how long each was insured. `vehicle_fraud.csv` contains only claims.

In this session, you will learn how a GLM estimates the average claim amount from several tariff features at once: as a base amount times one factor per feature.

- **Part 1**: From Group Means to Factors
- **Part 2**: The Gamma GLM
- **Part 3**: Predicting New Contracts

The explanations use small tables created directly in the code. All exercises use `vehicle_fraud.csv` from Notebook 03.

```
# Import required libraries
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf

# Avoid truncation of DataFrame output in Jupyter
pd.set_option('display.expand_frame_repr', False)

# Display all columns in the DataFrame output
pd.set_option('display.max_columns', None)

# Define a separator for better readability
sep = "=" * 50
```

Part 1: From Group Means to Factors

A tariff describes the expected claim amount as a **base amount** times one **factor** per feature, e.g. one factor for the vehicle type and one for the coverage type. The factors are estimated from past claims.

1.1 Mean Claim Amount per Group

`claims` holds 12 claims of compact and luxury cars with comprehensive or liability-only cover.

```
# Past claims of two vehicle types and two coverage types
claims = pd.DataFrame({
    'vehicle_type': ['Compact', 'Luxury', 'Compact', 'Luxury', 'Compact', 'Luxury',
                     'Compact', 'Luxury', 'Compact', 'Luxury', 'Compact', 'Luxury'],
    'coverage_type': ['Liability', 'Comprehensive', 'Comprehensive', 'Comprehensive',
        'Liability', 'Liability',
                      'Liability', 'Comprehensive', 'Comprehensive', 'Liability',
                          'Liability', 'Comprehensive'],
    'claim_amount': [2500.0, 8000.0, 5400.0, 10000.0, 3500.0, 4000.0,
                     2800.0, 8600.0, 6600.0, 5000.0, 3200.0, 9400.0]
})

print(f"\n{sep}\nPast Claims: claims\n{sep}")
print(claims)
```

```
==================================================
Past Claims: claims
==================================================
   vehicle_type  coverage_type  claim_amount
0       Compact      Liability        2500.0
1        Luxury  Comprehensive        8000.0
2       Compact  Comprehensive        5400.0
3        Luxury  Comprehensive       10000.0
4       Compact      Liability        3500.0
5        Luxury      Liability        4000.0
6       Compact      Liability        2800.0
7        Luxury  Comprehensive        8600.0
8       Compact  Comprehensive        6600.0
9        Luxury      Liability        5000.0
10      Compact      Liability        3200.0
11       Luxury  Comprehensive        9400.0
```

`groupby()` computes the mean claim amount per vehicle type (Notebook 03).

```
# Mean claim amount per vehicle type
vehicle_means = claims.groupby('vehicle_type')['claim_amount'].mean()

print(f"\n{sep}\nMean Claim Amount by Vehicle Type: claims.groupby('vehicle_type')
    ['claim_amount'].mean()\n{sep}")
print(vehicle_means)
```

```
==================================================
Mean Claim Amount by Vehicle Type: claims.groupby('vehicle_type')['claim_amount'].mean()
==================================================
vehicle_type
Compact    4000.0
Luxury     7500.0
Name: claim_amount, dtype: float64
```

1.2 Factors Relative to a Base Class

A factor compares a class with a **base class**. Here, the base class is `Compact`, the first class in alphabetical order. Each mean is divided by the mean of the base class:

vehicle_type	Mean claim amount	Factor
Compact	4,000	4,000 / 4,000 = 1.00
Luxury	7,500	7,500 / 4,000 = 1.875

The base class always has the factor 1. Factors computed from one feature at a time are **raw factors**.

```
# Raw factors relative to the base class Compact
vehicle_factors = vehicle_means / vehicle_means['Compact']

print(f"\n{sep}\nMean Claim Amount: vehicle_means\n{sep}")
print(vehicle_means)

print(f"\n{sep}\nRaw Factors: vehicle_means / vehicle_means['Compact']\n{sep}")
print(vehicle_factors.round(3))
```

```
================================================
Mean Claim Amount: vehicle_means
================================================
vehicle_type
Compact    4000.0
Luxury     7500.0
Name: claim_amount, dtype: float64

================================================
Raw Factors: vehicle_means / vehicle_means['Compact']
================================================
vehicle_type
Compact    1.000
Luxury     1.875
Name: claim_amount, dtype: float64
```

The same for `coverage_type`. Its base class is `Comprehensive`.

```
# Raw factors relative to the base class Comprehensive
coverage_means = claims.groupby('coverage_type')['claim_amount'].mean()
coverage_factors = coverage_means / coverage_means['Comprehensive']

print(f"\n{sep}\nMean Claim Amount by Coverage Type: claims.groupby('coverage_type')
    ['claim_amount'].mean()\n{sep}")
print(coverage_means)

print(f"\n{sep}\nRaw Factors: coverage_means / coverage_means['Comprehensive']\n{sep}")
print(coverage_factors.round(3))
```

```
================================================
Mean Claim Amount by Coverage Type: claims.groupby('coverage_type')['claim_amount'].mean()
================================================
coverage_type
Comprehensive    8000.0
Liability        3500.0
Name: claim_amount, dtype: float64

================================================
Raw Factors: coverage_means / coverage_means['Comprehensive']
================================================
coverage_type
Comprehensive    1.000
Liability        0.438
Name: claim_amount, dtype: float64
```

According to the raw factors, a luxury car costs 1.875 times as much as a compact car, and liability-only cover costs 0.44 times as much as comprehensive cover.

1.3 Two Features at Once

Raw factors are misleading when the features are related. First, count the claims per combination (Notebook 03):

```
# Number of claims per vehicle type and coverage type
print(f"\n{sep}\nGroup Sizes: claims.groupby(['vehicle_type', 'coverage_type'])
    ['claim_amount'].count().unstack()\n{sep}")
print(claims.groupby(['vehicle_type', 'coverage_type'])['claim_amount'].count().unstack())
```

```
================================================
Group Sizes: claims.groupby(['vehicle_type', 'coverage_type'])['claim_amount'].count().unstack()
================================================
coverage_type  Comprehensive  Liability
vehicle_type
Compact                    2          4
Luxury                     4          2
```

Luxury cars are mostly fully insured, compact cars mostly liability-only. Now the mean claim amount per combination:

```
# Mean claim amount per vehicle type and coverage type
print(f"\n{sep}\nMean Claim Amount: claims.groupby(['vehicle_type', 'coverage_type'])
    ['claim_amount'].mean().unstack()\n{sep}")
print(claims.groupby(['vehicle_type', 'coverage_type'])['claim_amount'].mean().unstack())
```

```
================================================
Mean Claim Amount: claims.groupby(['vehicle_type', 'coverage_type'])
['claim_amount'].mean().unstack()
================================================
coverage_type  Comprehensive  Liability
vehicle_type
Compact               6000.0     3000.0
Luxury                9000.0     4500.0
```

With the same coverage type, a luxury car costs 1.5 times as much as a compact car:

coverage_type	Compact	Luxury	Luxury / Compact
Comprehensive	6,000	9,000	9,000 / 6,000 = 1.5
Liability	3,000	4,500	4,500 / 3,000 = 1.5

The raw factor 1.875 is too high: luxury cars are more often fully insured, so their mean also contains the higher claim amounts of comprehensive cover. In the same way, the raw factor 0.44 for liability-only cover is too low; with the same vehicle type, it is 0.5. A GLM estimates all factors at once and separates the two effects.

> **Exercise 1: Raw Factors on `vehicle_fraud.csv`**
>
> Compute raw factors for vehicle type and coverage type and check how the two features are related.

Fraudulent claims would distort the average claim amount of honest customers, so all exercises use only legitimate claims. The first three code cells prepare the data: they remove the duplicates, add the age groups from Notebook 03 and keep the legitimate claims with a known age.

```
# Load the dataset and remove the duplicates
df = pd.read_csv('vehicle_fraud.csv')
df = df.drop_duplicates()
```

```
# Age groups as in Notebook 03
df['age_group'] = pd.cut(df['policyholder_age'], bins=[0, 30, 40, 50, 100], right=False,
                         labels=['<30', '30-39', '40-49', '50+'])
```

```
# Legitimate claims with a known age (the placeholder 0 means unknown)
legit_claims = df[(df['fraud'] == 0) & (df['policyholder_age'] >= 18)]
```

Task 1: Compute the mean claim amount per `vehicle_type` and the raw factors relative to `Compact`. Which vehicle type has the highest raw factor?

```
# Task 1: Mean claim amount per vehicle type
means_by_type = legit_claims.groupby('vehicle_type')['claim_amount'].mean()

print(f"\n{sep}\nMean Claim Amount by Vehicle Type: legit_claims.groupby('vehicle_type')
    ['claim_amount'].mean()\n{sep}")
print(means_by_type.round(2))
```

```
================================================
Mean Claim Amount by Vehicle Type: legit_claims.groupby('vehicle_type')['claim_amount'].mean()
================================================
vehicle_type
Compact        7444.90
Luxury        12409.47
Mid-range      8482.78
SUV           10433.39
Sports car    11221.66
Name: claim_amount, dtype: float64
```

```
# Task 1: Raw factors relative to Compact
raw_type_factors = means_by_type / means_by_type['Compact']

print(f"\n{sep}\nRaw Factors: means_by_type / means_by_type['Compact']\n{sep}")
print(raw_type_factors.round(3))
```

```
================================================
Raw Factors: means_by_type / means_by_type['Compact']
================================================
vehicle_type
Compact       1.000
Luxury        1.667
Mid-range     1.139
SUV           1.401
Sports car    1.507
Name: claim_amount, dtype: float64
```

Task 2: Compute the raw factors of `coverage_type` relative to `Comprehensive`.

```
# Task 2: Mean claim amount per coverage type
means_by_coverage = legit_claims.groupby('coverage_type')['claim_amount'].mean()

print(f"\n{sep}\nMean Claim Amount by Coverage Type: legit_claims.groupby('coverage_type')
    ['claim_amount'].mean()\n{sep}")
print(means_by_coverage.round(2))
```

```
==================================================
Mean Claim Amount by Coverage Type: legit_claims.groupby('coverage_type')['claim_amount'].mean()
==================================================
coverage_type
Comprehensive       11463.25
Liability            7907.31
Partial coverage     8759.79
Name: claim_amount, dtype: float64
```

```
# Task 2: Raw factors relative to Comprehensive
raw_coverage_factors = means_by_coverage / means_by_coverage['Comprehensive']

print(f"\n{sep}\nRaw Factors: means_by_coverage /
    means_by_coverage['Comprehensive']\n{sep}")
print(raw_coverage_factors.round(3))
```

```
==================================================
Raw Factors: means_by_coverage / means_by_coverage['Comprehensive']
==================================================
coverage_type
Comprehensive       1.000
Liability           0.690
Partial coverage    0.764
Name: claim_amount, dtype: float64
```

Task 3: Count the claims per `vehicle_type` and `coverage_type`. Which vehicle types are mostly fully insured?

```
# Task 3: Number of claims per vehicle type and coverage type
print(f"\n{sep}\nGroup Sizes: legit_claims.groupby(['vehicle_type', 'coverage_type'])
    ['claim_amount'].count().unstack()\n{sep}")
print(legit_claims.groupby(['vehicle_type', 'coverage_type'])
    ['claim_amount'].count().unstack())
```

```
==================================================
Group Sizes: legit_claims.groupby(['vehicle_type', 'coverage_type'])
['claim_amount'].count().unstack()
==================================================
coverage_type  Comprehensive  Liability  Partial coverage
vehicle_type
Compact                  603        549               596
Luxury                  1251        269               238
Mid-range                739        526               530
SUV                      945        387               426
Sports car              1112        324               293
```

Task 4: Compute the mean claim amount per `vehicle_type` and `coverage_type`. With `Liability` cover, how many times as much does a luxury claim cost as a compact claim? Compare with the raw factor from Task 1.

```
# Task 4: Mean claim amount per vehicle type and coverage type
type_coverage_means = legit_claims.groupby(['vehicle_type', 'coverage_type'])
    ['claim_amount'].mean().unstack()

print(f"\n{sep}\nMean Claim Amount: legit_claims.groupby(['vehicle_type', 'coverage_type'])
    ['claim_amount'].mean().unstack()\n{sep}")
print(type_coverage_means.round(2))
```

```
==================================================
Mean Claim Amount: legit_claims.groupby(['vehicle_type', 'coverage_type'])
['claim_amount'].mean().unstack()
==================================================
coverage_type  Comprehensive  Liability  Partial coverage
vehicle_type
Compact              8618.31    6469.38           7156.29
Luxury              13372.88   10148.62           9900.79
Mid-range            9482.49    7220.59           8341.52
SUV                 11193.39    8704.67          10317.95
Sports car          12403.30    8645.42           9585.87
```

```
# Task 4: Luxury vs. Compact with Liability cover
print(f"\n{sep}\nLuxury / Compact with Liability: type_coverage_means.loc['Luxury',
    'Liability'] / type_coverage_means.loc['Compact', 'Liability']\n{sep}")
print(round(type_coverage_means.loc['Luxury', 'Liability'] /
    type_coverage_means.loc['Compact', 'Liability'], 3))
```

```
==================================================
Luxury / Compact with Liability: type_coverage_means.loc['Luxury', 'Liability'] /
type_coverage_means.loc['Compact', 'Liability']
==================================================
1.569
```

Luxury cars (71%), sports cars (64%) and SUVs (54%) are mostly fully insured, compact cars only in 34% of the claims. On average, a luxury claim costs 1.67 times as much as a compact claim. With the same `Liability` cover, it is only 1.57 times as much: the raw factor also contains the higher coverage of luxury cars.

Part 2: The Gamma GLM

A GLM (generalized linear model) estimates all factors at once. Logistic regression from Notebook 07 is a GLM for 0/1 targets. For claim amounts, a GLM needs a different distribution and link.

2.1 Distribution and Link

Choice	For claim amounts	Reason
Distribution	Gamma	Claim amounts are positive and right-skewed (Notebook 04)
Link	Log	The factors multiply

With the log link, the logarithm of the expected claim amount is a sum of coefficients β:

$$\log(\text{expected claim amount}) = \beta_0 + \beta_{\text{Luxury}} + \beta_{\text{Liability}}$$

The exponential function turns the sum into a product:

$$\text{expected claim amount} = e^{\beta_0} \cdot e^{\beta_{\text{Luxury}}} \cdot e^{\beta_{\text{Liability}}}$$

e^{β_0} is the **base amount** of the base classes, every other e^{β} is a **factor**. A claim only gets the coefficients of its own classes: for a compact car, β_{Luxury} is left out.

2.2 Fitting with `smf.glm()`

`smf.glm()` describes the model with a **formula**: the target left of `~`, the features right of it, joined with `+`. Text columns become classes automatically, with the first class in alphabetical order as the base class. `family=` sets the distribution and the link. `fit()` estimates the coefficients.

```
# Gamma GLM with a log link
glm = smf.glm('claim_amount ~ vehicle_type + coverage_type', data=claims,
              family=sm.families.Gamma(link=sm.families.links.Log())).fit()
print('GLM fitted.')
```

```
GLM fitted.
```

2.3 Reading `summary()`

`summary()` shows the fitted model. The upper table repeats the setup: 12 claims, family `Gamma`, link `Log`. The lower table has one row per coefficient: `Intercept` is β_0, `vehicle_type[T.Luxury]` is the coefficient of `Luxury` compared with the base class `Compact`. The column `coef` holds the coefficients, the other columns show how precisely they are estimated.

```
# Summary of the fitted model
print(f"\n{sep}\nSummary: glm.summary()\n{sep}")
print(glm.summary())
```

```
==================================================
Summary: glm.summary()
==================================================
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:           claim_amount   No. Observations:                   12
Model:                            GLM   Df Residuals:                        9
Model Family:                   Gamma   Df Model:                            2
Link Function:                    Log   Scale:                          0.015309
Method:                          IRLS   Log-Likelihood:                -93.105
Date:                Fri, 30 Oct 2026   Deviance:                      0.13900
Time:                        15:15:03   Pearson chi2:                    0.138
No. Iterations:                     6   Pseudo R-squ. (CS):              1.000
Covariance Type:            nonrobust
================================================================================================
                                   coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------------------------
Intercept                        8.6995      0.071    121.783      0.000       8.560       8.840
vehicle_type[T.Luxury]           0.4055      0.076      5.351      0.000       0.257       0.554
coverage_type[T.Liability]      -0.6931      0.076     -9.148      0.000      -0.842      -0.545
================================================================================================
```

2.4 Base Amount and Factors

`glm.params` holds the coefficients. `np.exp()` turns them into the base amount and the factors.

```
# Base amount and factors
factors = np.exp(glm.params)

print(f"\n{sep}\nCoefficients: glm.params\n{sep}")
print(glm.params.round(4))

print(f"\n{sep}\nBase Amount and Factors: np.exp(glm.params)\n{sep}")
print(factors.round(3))
```

```
==================================================
Coefficients: glm.params
==================================================
Intercept                     8.6995
vehicle_type[T.Luxury]        0.4055
coverage_type[T.Liability]   -0.6931
dtype: float64

==================================================
Base Amount and Factors: np.exp(glm.params)
==================================================
Intercept                     6000.0
vehicle_type[T.Luxury]           1.5
coverage_type[T.Liability]       0.5
dtype: float64
```

A compact car with comprehensive cover has an expected claim amount of 6,000 €. A luxury car costs 1.5 times as much, liability-only cover half as much.

2.5 GLM Factors vs. Raw Factors

```
# Raw factors and GLM factors side by side
comparison = pd.DataFrame({
    'raw': [vehicle_factors['Luxury'], coverage_factors['Liability']],
    'glm': [factors['vehicle_type[T.Luxury]'], factors['coverage_type[T.Liability]']]
}, index=['Luxury', 'Liability'])

print(f"\n{sep}\nRaw Factors vs. GLM Factors: comparison\n{sep}")
print(comparison.round(3))
```

```
==================================================
Raw Factors vs. GLM Factors: comparison
==================================================
             raw  glm
Luxury     1.875  1.5
Liability  0.438  0.5
```

The GLM finds the factors 1.5 and 0.5 from Section 1.3. The raw factors mix the two effects; the GLM separates them because it estimates all factors at once.

> **Exercise 2: Gamma GLM on `vehicle_fraud.csv`**
>
> Fit a Gamma GLM with four tariff features, read its factors and compare them with the raw factors.
>
> Continue with `legit_claims` from Exercise 1.

Task 1: Fit a Gamma GLM `glm_model` with a log link: target `claim_amount`, features `vehicle_type`, `coverage_type`, `region` and `age_group`. Show its summary.

```
# Task 1: Gamma GLM with four features
glm_model = smf.glm('claim_amount ~ vehicle_type + coverage_type + region + age_group',
    data=legit_claims,
                    family=sm.families.Gamma(link=sm.families.links.Log())).fit()
print('GLM fitted.')
```

```
GLM fitted.
```

```
# Task 1: Summary
print(f"\n{sep}\nSummary: glm_model.summary()\n{sep}")
print(glm_model.summary())
```

```
================================================
Summary: glm_model.summary()
================================================
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:           claim_amount   No. Observations:                 8788
Model:                            GLM   Df Residuals:                     8769
Model Family:                   Gamma   Df Model:                           18
Link Function:                    Log   Scale:                          0.80651
Method:                          IRLS   Log-Likelihood:                -88884.
Date:                Fri, 30 Oct 2026   Deviance:                       6408.5
Time:                        15:15:04   Pearson chi2:                 7.07e+03
No. Iterations:                    11   Pseudo R-squ. (CS):             0.1383
Covariance Type:            nonrobust
=============================================================================================================
                                      coef    std err          z      P>|z|      [0.025      0.975]
-------------------------------------------------------------------------------------------------------------
Intercept                             9.9375      0.093    106.944      0.000       9.755      10.120
vehicle_type[T.Luxury]                0.4262      0.031     13.647      0.000       0.365       0.487
vehicle_type[T.Mid-range]             0.1317      0.030      4.360      0.000       0.073       0.191
vehicle_type[T.SUV]                   0.2910      0.031      9.510      0.000       0.231       0.351
vehicle_type[T.Sports car]            0.3254      0.031     10.477      0.000       0.265       0.386
coverage_type[T.Liability]           -0.2853      0.024    -11.710      0.000      -0.333      -0.238
coverage_type[T.Partial coverage]    -0.1663      0.024     -6.818      0.000      -0.214      -0.119
region[T.PLZ_02]                     -0.1941      0.043     -4.505      0.000      -0.279      -0.110
region[T.PLZ_03]                     -0.1928      0.042     -4.561      0.000      -0.276      -0.110
region[T.PLZ_04]                     -0.2790      0.043     -6.469      0.000      -0.364      -0.194
region[T.PLZ_05]                     -0.3236      0.043     -7.600      0.000      -0.407      -0.240
region[T.PLZ_06]                     -0.2925      0.043     -6.876      0.000      -0.376      -0.209
region[T.PLZ_07]                     -0.3913      0.042     -9.216      0.000      -0.475      -0.308
region[T.PLZ_08]                     -0.3927      0.043     -9.068      0.000      -0.478      -0.308
region[T.PLZ_09]                     -0.4475      0.043    -10.502      0.000      -0.531      -0.364
region[T.PLZ_10]                     -0.4519      0.042    -10.774      0.000      -0.534      -0.370
age_group[T.30-39]                   -0.1997      0.090     -2.210      0.027      -0.377      -0.023
age_group[T.40-49]                   -0.4835      0.087     -5.544      0.000      -0.654      -0.313
age_group[T.50+]                     -0.8117      0.087     -9.365      0.000      -0.982      -0.642
=============================================================================================================
```

Task 2: Compute the base amount and the factors `glm_factors`. Which classes are the base classes? Which vehicle type, coverage type, region and age group has the highest expected claim amount?

```
# Task 2: Base amount and factors
glm_factors = np.exp(glm_model.params)

print(f"\n{sep}\nBase Amount and Factors: np.exp(glm_model.params)\n{sep}")
print(glm_factors.round(3))
```

```
================================================
Base Amount and Factors: np.exp(glm_model.params)
================================================
Intercept                             20691.344
vehicle_type[T.Luxury]                    1.531
vehicle_type[T.Mid-range]                 1.141
vehicle_type[T.SUV]                       1.338
vehicle_type[T.Sports car]                1.385
coverage_type[T.Liability]                0.752
coverage_type[T.Partial coverage]         0.847
region[T.PLZ_02]                          0.824
region[T.PLZ_03]                          0.825
region[T.PLZ_04]                          0.757
region[T.PLZ_05]                          0.724
region[T.PLZ_06]                          0.746
region[T.PLZ_07]                          0.676
region[T.PLZ_08]                          0.675
region[T.PLZ_09]                          0.639
region[T.PLZ_10]                          0.636
age_group[T.30-39]                        0.819
age_group[T.40-49]                        0.617
age_group[T.50+]                          0.444
dtype: float64
```

The base classes `Compact`, `Comprehensive`, `PLZ_01` and `<30` do not appear in the table. A compact car with comprehensive cover in `PLZ_01` and a driver under 30 has an expected claim amount of 20,691 €. All other factors of `coverage_type`, `region` and `age_group` are below 1, so there the base classes are the most expensive ones. Among the vehicle types, `Luxury` has the highest factor (1.53). The expected claim amount falls with age: drivers aged 50 and over cost 0.44 times as much as drivers under 30.

Task 3: Compare the GLM factors of `Luxury`, `Sports car` and `Liability` with the raw factors from Exercise 1. Where do they differ most? Why?

```
# Task 3: Raw factors vs. GLM factors
factor_comparison = pd.DataFrame({
    'raw': [raw_type_factors['Luxury'], raw_type_factors['Sports car'],
        raw_coverage_factors['Liability']],
    'glm': [glm_factors['vehicle_type[T.Luxury]'], glm_factors['vehicle_type[T.Sports
        car]'],
            glm_factors['coverage_type[T.Liability]']]
}, index=['Luxury', 'Sports car', 'Liability'])

print(f"\n{sep}\nRaw Factors vs. GLM Factors: factor_comparison\n{sep}")
print(factor_comparison.round(3))
```

```
================================================
Raw Factors vs. GLM Factors: factor_comparison
================================================
              raw    glm
Luxury      1.667  1.531
Sports car  1.507  1.385
Liability   0.690  0.752
```

The raw factors overstate the vehicle types that are mostly fully insured: 1.67 instead of 1.53 for `Luxury`, 1.51 instead of 1.39 for `Sports car`. The raw factor of `Liability` is too low (0.69 instead of 0.75) because liability-only contracts mostly belong to cheaper cars. The GLM separates these effects.

Part 3: Predicting New Contracts

3.1 `predict()`

`predict()` takes a DataFrame with the feature columns of the formula and returns the expected claim amount per row. The text columns need no preparation: the formula turns them into classes as in `fit()`.

```
# Three new contracts
new_contracts = pd.DataFrame({
    'vehicle_type': ['Compact', 'Luxury', 'Luxury'],
    'coverage_type': ['Comprehensive', 'Comprehensive', 'Liability']
})

print(f"\n{sep}\nNew Contracts: new_contracts\n{sep}")
print(new_contracts)
```

```
================================================
New Contracts: new_contracts
================================================
  vehicle_type  coverage_type
0      Compact  Comprehensive
1       Luxury  Comprehensive
2       Luxury     Liability
```

```python
# Expected claim amount per contract
predictions = glm.predict(new_contracts)

print(f"\n{sep}\nExpected Claim Amounts: glm.predict(new_contracts)\n{sep}")
print(predictions.round(2))
```

```
================================================
Expected Claim Amounts: glm.predict(new_contracts)
================================================
0    6000.0
1    9000.0
2    4500.0
dtype: float64
```

3.2 Checking a Prediction by Hand

The expected claim amount is the base amount times the factors of the contract's classes. Contract 2 is a luxury car with liability-only cover:

Step	Value
Base amount (Compact, Comprehensive)	6,000
× factor Luxury	1.5
× factor Liability	0.5
= expected claim amount	4,500

```python
# Base amount times the factors of contract 2
print(f"\n{sep}\nBase Amount and Factors: factors\n{sep}")
print(factors.round(3))

print(f"\n{sep}\nContract 2 by Hand: factors['Intercept'] *
    factors['vehicle_type[T.Luxury]'] * factors['coverage_type[T.Liability]']\n{sep}")
print(round(factors['Intercept'] * factors['vehicle_type[T.Luxury]'] *
    factors['coverage_type[T.Liability]'], 2))
```

```
================================================
Base Amount and Factors: factors
================================================
Intercept                     6000.0
vehicle_type[T.Luxury]           1.5
coverage_type[T.Liability]       0.5
dtype: float64

================================================
Contract 2 by Hand: factors['Intercept'] * factors['vehicle_type[T.Luxury]'] *
factors['coverage_type[T.Liability]']
================================================
4500.0
```

Exercise 3: Predicting New Contracts

Predict the expected claim amount of three new contracts and check one prediction by hand.

Continue with `glm_model` and `glm_factors` from Exercise 2.

Task 1: Create `new_policies` with three contracts and predict their expected claim amounts.

Contract	vehicle_type	coverage_type	region	age_group
0	Compact	Liability	PLZ_10	50+
1	SUV	Partial coverage	PLZ_05	30-39
2	Luxury	Comprehensive	PLZ_01	<30

```
# Task 1: Three new contracts
new_policies = pd.DataFrame({
    'vehicle_type': ['Compact', 'SUV', 'Luxury'],
    'coverage_type': ['Liability', 'Partial coverage', 'Comprehensive'],
    'region': ['PLZ_10', 'PLZ_05', 'PLZ_01'],
    'age_group': ['50+', '30-39', '<30']
})

print(f"\n{sep}\nNew Contracts: new_policies\n{sep}")
print(new_policies)
```

```
================================================
New Contracts: new_policies
================================================
  vehicle_type     coverage_type  region age_group
0      Compact         Liability  PLZ_10       50+
1          SUV  Partial coverage  PLZ_05     30-39
2       Luxury     Comprehensive  PLZ_01       <30
```

```
# Task 1: Expected claim amounts
predicted = glm_model.predict(new_policies)

print(f"\n{sep}\nExpected Claim Amounts: glm_model.predict(new_policies)\n{sep}")
print(predicted.round(2))
```

```
================================================
Expected Claim Amounts: glm_model.predict(new_policies)
================================================
0     4396.23
1    13889.68
2    31687.90
dtype: float64
```

Contract 2 has the base classes of `coverage_type`, `region` and `age_group`, so its expected claim amount is the base amount times the factor of `Luxury` alone.

Task 2: Check the prediction of contract 1 by hand: base amount times its four factors.

```
# Task 2: Contract 1 by hand
by_hand = (glm_factors['Intercept'] * glm_factors['vehicle_type[T.SUV]'] *
    glm_factors['coverage_type[T.Partial coverage]']
           * glm_factors['region[T.PLZ_05]'] * glm_factors['age_group[T.30-39]'])

print(f"\n{sep}\nContract 1 by Hand: base amount * factors of SUV, Partial coverage, PLZ_05,
    30-39\n{sep}")
print(round(by_hand, 2))
```

```
================================================
Contract 1 by Hand: base amount * factors of SUV, Partial coverage, PLZ_05, 30-39
================================================
13889.68
```

The base amount times the four factors gives 13,889.68 €, exactly the prediction of `predict()`.

Task 3: Contract 0 switches from `Liability` to `Comprehensive`. Predict its new expected claim amount. By which factor does it change? Compare with the factor of `Liability`.

```
# Task 3: Contract 0 with comprehensive cover
upgraded = pd.DataFrame({'vehicle_type': ['Compact'], 'coverage_type': ['Comprehensive'],
                         'region': ['PLZ_10'], 'age_group': ['50+']})
new_amount = glm_model.predict(upgraded)[0]

print(f"\n{sep}\nExpected Claim Amount with Comprehensive Cover: glm_model.predict(upgraded)
    [0]\n{sep}")
print(round(new_amount, 2))
```

```
================================================
Expected Claim Amount with Comprehensive Cover: glm_model.predict(upgraded)[0]
================================================
5847.61
```

```
# Task 3: Change factor and factor of Liability
print(f"\n{sep}\nChange: new_amount / predicted[0]\n{sep}")
print(round(new_amount / predicted[0], 3))

print(f"\n{sep}\nInverse Factor of Liability: 1 /
    glm_factors['coverage_type[T.Liability]']\n{sep}")
print(round(1 / glm_factors['coverage_type[T.Liability]'], 3))
```

```
================================================
Change: new_amount / predicted[0]
================================================
1.33

================================================
Inverse Factor of Liability: 1 / glm_factors['coverage_type[T.Liability]']
================================================
1.33
```

With comprehensive cover, the expected claim amount of contract 0 rises from 4,396 € to 5,848 €: by the factor 1.33 = 1 / 0.752. Changing one class multiplies the expected claim amount by the ratio of the two factors; all other factors stay the same.

Summary

- **Factors:** group means relative to a base class; raw factors mix the effects of related features
- **Gamma GLM:** `smf.glm()` with a formula and `family=sm.families.Gamma(link=sm.families.links.Log())`, `summary()`, base amount and factors with `np.exp()` of the coefficients
- **Predicting:** `predict()` on new contracts; expected claim amount = base amount × factors

The expected claim amount is not yet a premium: a premium also needs the claim frequency.

Next: detecting fraudulent claims in Notebook 09.

- statsmodels: Generalized Linear Models